

Analisis de Patrones y Arquetipos - SmartLogix

1. Contexto del problema

SmartLogix busca resolver problemas tipicos de una PYME de eCommerce: inventario que cambia constantemente, pedidos que dependen del stock disponible y envios que deben ser trazables. Un enfoque monolitico acoplaria todas las reglas en un solo backend y dificultaria la mantencion. Por eso se usa una arquitectura con frontend React, BFF y microservicios.

2. Patrones de diseno implementados

2.1 Backend For Frontend (BFF)

Componente: bff.

Problema que resuelve: el frontend necesitaria llamar directamente a inventario, pedidos y envios. Eso aumenta acoplamiento y obliga al cliente a conocer la topologia interna.

Aplicacion: BffController expone /api/bff/dashboard, /api/bff/inventario, /api/bff/pedidos y /api/bff/envios. BffService consume internamente los microservicios y devuelve una respuesta adaptada al frontend.

Aporte: reduce llamadas desde React, centraliza errores, estandariza el contrato HTTP y facilita cambiar microservicios sin romper la interfaz.

2.2 API Gateway

Componente: bff.

Problema que resuelve: multiples puntos de entrada desde el frontend y exposicion directa de microservicios.

Aplicacion: todas las operaciones CRUD y cambios de estado pasan por /api/bff/*. El frontend solo conoce el BFF.

Aporte: simplifica seguridad futura, trazabilidad, CORS y control de version de API.

2.3 Repository Pattern

Componentes: ms-inventario, ms-pedidos, ms-envios.

Problema que resuelve: mezclar reglas de negocio con consultas SQL o acceso a base de datos.

Aplicacion: ProductoRepository, PedidoRepository y EnvioRepository extienden JpaRepository. Los servicios usan estos repositorios como abstraccion.

Aporte: mejora mantenibilidad, permite probar servicios con mocks y separa persistencia de reglas de negocio.

2.4 DTO

Componentes: todos los servicios backend.

Problema que resuelve: exponer entidades JPA directamente puede filtrar campos internos y acoplar la API al modelo de persistencia.

Aplicacion: ProductoDTO, PedidoDTO, EnvioDTO y DashboardDTO son contratos de entrada y salida.

Aporte: define datos exactos para el cliente, facilita validaciones y evita exponer detalles internos.

2.5 Factory Method

Componente: ms-pedidos, clase PedidoFactory.

Problema que resuelve: crear pedidos con estados y atributos iniciales repetidos en varias partes del sistema.

Aplicacion: la fabrica crea pedidos en estado CREADO, VALIDADO o APROBADO segun el flujo de negocio.

Aporte: centraliza la logica de creacion, reduce duplicidad y evita inconsistencias de estado.

2.6 Circuit Breaker manual

Componente: ms-pedidos, clase PedidoService.

Problema que resuelve: si inventario falla, pedidos podria quedar bloqueado o provocar cascada de fallos.

Aplicacion: PedidoService abre un circuito cuando falla la consulta a ms-inventario. En ese caso registra el pedido como pendiente en vez de hacer fallar todo el flujo.

Aporte: mejora resiliencia y permite degradacion controlada.

2.7 Custom Hook y Service Layer en frontend

Componentes: frontend/src/hooks/useDashboard.js y frontend/src/services/api.js.

Problema que resuelve: duplicacion de logica de carga HTTP, estados de loading y gestion de errores dentro de los componentes visuales.

Aplicacion: api.js centraliza Axios contra el BFF, y useDashboard encapsula carga del dashboard.

Aporte: componentes mas limpios, pruebas unitarias mas simples y menor acoplamiento.

3. Patrones arquitectonicos

3.1 Microservicios

Cada dominio se separa en un servicio independiente: inventario, pedidos y envios. Esto permite escalar, mantener y probar cada parte por separado.

3.2 Arquitectura por capas

Cada microservicio mantiene estructura consistente: controller, service, repository, model, dto. Esto evita mezclar responsabilidades.

3.3 BFF + Microservicios

El BFF evita que el frontend dependa de todos los microservicios. Los microservicios conservan su independencia, mientras el BFF entrega una API optimizada para React.

4. Arquetipos Maven

Se agregaron dos arquetipos en arquetipos-maven:

- smartlogix-ms-archetype: genera microservicios con controller, service, repository, model, dto, test, pom.xml y configuracion de JaCoCo.
- smartlogix-bff-archetype: genera un BFF con controller, service, dto, RestTemplate, test, pom.xml y configuracion de JaCoCo.

Estos arquetipos aseguran coherencia porque cada nuevo backend nace con la misma estructura, mismas dependencias base y misma convencion de pruebas.

5. Justificacion de escalabilidad y mantenibilidad

La solucion es escalable porque cada microservicio puede crecer y desplegarse de forma independiente. Es mantenible porque las reglas de negocio se ubican en servicios, el acceso a datos en repositorios, la API se define con DTO y el frontend no conoce la red interna gracias al BFF.

6. Evidencia en codigo

- BFF y API Gateway: bff/src/main/java/.../controller/BffController.java y BffService.java.
- Repository Pattern: ProductoRepository, PedidoRepository, EnvioRepository.
- Factory Method: ms-pedidos/src/main/java/.../factory/PedidoFactory.java.
- Circuit Breaker: ms-pedidos/src/main/java/.../service/PedidoService.java.
- DTO: carpetas dto de cada backend.
- Frontend Service Layer: frontend/src/services/api.js.
- Custom Hook: frontend/src/hooks/useDashboard.js.